

A Resource-Constrained Dual-SLM Pipeline for Continual Synthetic-Data SFT

An implementation study and evaluation protocol

Krrish Rebbawar, Pranav Chandane, and Jayosmit Shinde

Applied research prototype | 2026

Abstract

Continual adaptation can improve a language model on incoming tasks while degrading performance on earlier tasks, a stability-plasticity problem commonly called catastrophic forgetting. This paper describes a resource-constrained dual-small-language-model (Dual-SLM) prototype for continual synthetic-data supervised fine-tuning (SFT). A frozen SmolLM2-360M-Instruct teacher generates responses; a heuristic entropy score (HES) filters examples; a small set of rule-based prompt rewrites transforms prompts; and a SmolLM2-135M student is updated through a persistent rank-8 LoRA adapter while its base weights remain frozen in standard runs. We conducted a documentary and static audit of the implementation, case studies, and available trainer artifacts. Checkpoints substantiate training activity for Case Studies 1-11 and 13, including a CS10 full-fine-tuning run whose recorded loss peaked at approximately 77.27 and ended at approximately 23.70. Historical retention and perplexity claims cannot be independently reproduced from the current snapshot: the fixed evaluation set contains only three algebra and two empathy examples, and no saved evaluation-results file is present. The work therefore establishes implementation feasibility and a research protocol, not evidence that the architecture mitigates or eliminates catastrophic forgetting. Its contribution is an inspectable low-resource pipeline and a controlled evaluation design for testing whether teacher-student decoupling and parameter-efficient updates improve retention.

Keywords

continual learning; catastrophic forgetting; synthetic-data SFT; small language models; LoRA; teacher-student learning

1. Introduction

Language models are usually trained on fixed corpora, but deployed systems may need to incorporate new tasks, facts, or preferences over time. Continual learning seeks to add capability while retaining useful prior behavior. This is difficult because gradient updates for a new data distribution can interfere with parameters used by earlier tasks [1]. The problem is studied in continual pretraining, domain adaptation, and continual fine-tuning of language models [2].

The 2025 SEAL work studies models that generate self-edits and apply persistent SFT updates, with a reinforcement-learning loop that rewards downstream task performance [3]. Its experiments include a particular sequential knowledge-incorporation setting in which earlier-task performance gradually degrades; the paper does not establish that single-model self-editing inevitably collapses. It also discusses a teacher-student formulation as one possible way to decouple generation from learning [3]. This motivates investigating a dual-model pipeline, but the present project is not an implementation or matched replication of SEAL.

This project began as a laptop-scale engineering prototype. A frozen teacher generates synthetic responses, a heuristic gate screens them, and a smaller student is adapted through LoRA. The proposed rationale is that the teacher remains unchanged and the student base weights remain fixed, which may limit interference with pretrained base parameters. However, persistent adapter weights continue to change and can themselves forget earlier adapter behavior. Frozen base weights alone do not demonstrate system-level retention.

The research problem is therefore framed as an empirical question: under matched sequential-learning conditions, does a frozen-teacher, HES-filtered, persistent-LoRA pipeline retain earlier-task performance better than suitable single-model or adapter baselines? The existing artifacts do not answer this question. They support an implementation study and a prospective hypothesis.

1.1 Motivation and significance

A small-model pipeline can be useful where GPU memory, compute time, or deployment constraints limit experimentation. The project operationalizes a teacher-student workflow with quantized model loading, a prompt-level entropy gate, and a persistent low-rank student update. These components make a testable design. Their value for continual learning must be determined with retention measurements, not inferred from the architecture or from falling training loss.

1.2 Research contribution

This paper contributes (i) a code-level description of the implemented pipeline and its exceptions; (ii) an evidence-bounded account of what the saved trainer artifacts establish; (iii) a correction of case-study claims that exceed available evidence; and (iv) a controlled evaluation protocol that would test the central retention hypothesis.

2. Literature Review

2.1 Continual learning and forgetting

Continual learning studies adaptation to a sequence of changing tasks or domains while limiting degradation on earlier tasks. Surveys of continual learning for language models organize methods around continual pretraining, domain-adaptive pretraining, and continual fine-tuning, and emphasize evaluation across new and previously learned tasks [2]. Common strategies include replay, regularization, parameter isolation, and distillation. These approaches expose a trade-off: retaining past examples or dedicating parameters can improve stability but may add memory, compute, or inference costs.

2.2 Self-adaptation and SEAL

SEAL uses a model to produce self-edits that can reshape training examples, specify update directives, and invoke tools. The resulting model is evaluated downstream, and reinforcement learning optimizes the self-edit policy [3]. In its sequential knowledge-incorporation experiment, SEAL reports gradual decline on earlier tasks under a setup without an explicit retention objective; the authors describe the decline as incomplete and identify retention as an open direction. They also discuss teacher-student decoupling as a possible formulation [3]. The present system differs materially: it uses a separate fixed teacher, scripted prompts, threshold-based entropy filtering, and conventional SFT; it does not learn an edit policy with downstream rewards.

2.3 Parameter-efficient adaptation

LoRA freezes pretrained weights and introduces trainable low-rank matrices into selected layers [4]. This reduces trainable parameters and memory needs relative to full fine-tuning. Quantized LoRA methods such as QLoRA combine low-bit base-model storage with adapter training to further reduce memory requirements [5]. These methods address update efficiency and protect base parameters, but protection of the base does not ensure that one continually updated adapter retains earlier behavior. The adapter remains mutable and must be evaluated over the task sequence.

2.4 Small models and synthetic data

SmolLM2 is a family of compact models that includes 135M and 360M parameter variants [6]. The project code uses SmolLM2-360M-Instruct as teacher and SmolLM2-135M as student. The prototype generates a small number of responses in memory from hard-coded prompts; it does not train on the large pretraining corpus used to create SmolLM2, nor does the execution path load the externally linked GSM8K or SQuAD datasets. The project should therefore be described as synthetic-data SFT using small prompt sets, rather than corpus-scale training.

2.5 Research gap

Existing work establishes the importance of continual-learning evaluation and offers multiple retention mechanisms, while SEAL motivates self-editing and discusses decoupling generation from learning. The gap addressed here is narrow: a low-resource implementation that combines a static teacher, an entropy-based heuristic gate, prompt rewriting, and a persistent student adapter. The project has not shown whether this combination improves retention against matched alternatives. The required evidence is a controlled task stream with scores after each update, repeated seeds, and baselines matched for data and optimization budgets.

3. Research Objectives and Questions

The study has three objectives:

Document the implemented dual-SLM synthetic-data SFT pipeline, including model sizes, HES rule, LoRA configuration, and known deviations.

Determine what claims about execution and model behavior are supported by saved code, case studies, and trainer artifacts.

Specify an evaluation protocol capable of testing whether the architecture reduces forgetting relative to matched sequential-update baselines.

Research questions:

RQ1: Can the teacher generation, governance, and student-update stages be executed as a low-resource synthetic-data SFT prototype using sequential model loading?

RQ2: Do the available artifacts demonstrate improved retention on previously learned tasks after subsequent updates?

RQ3: What controlled comparisons and measurements are required to test the hypothesis that this design mitigates forgetting?

4. Proposed Methodology

4.1 Research design

This manuscript uses an implementation-study and artifact-audit design. The primary analysis is static: project source code, case-study documents, master data log, checkpoint directories, and trainer-state JSON files were inspected. No historical training run was repeated for this paper, no model weights were newly evaluated, and no participant data were collected. Accordingly, this paper reports code and artifact evidence and does not treat unverified case-study prose as experimental measurements.

4.2 System and procedure

The standard path consists of four stages (Figure 1). First, task prompts are passed to SmolLM2-360M-Instruct, loaded in 4-bit NF4 with FP16 computation. Generation requests at most 128 new tokens and stores each generated response with per-step score vectors. The teacher is deleted and memory cleanup is requested before student training. Second, the governance function computes softmax predictive entropy from each generation-step score vector using base-2 logarithms. For T generated tokens, it sorts token entropies and sums the highest 20% using the code-derived cutoff index; the standard path retains an example when HES is at least 10.0. Third, a small set of literal prefix substitutions rewrites the prompt; for example, an input beginning “Explain ” is transformed with the implemented “Describe in simple terms about ” prefix. Unmatched prompts receive a fixed “Can you give more details about” wrapper. Fourth, the generated answer and rewritten prompt are formatted for SFT of SmolLM2-135M.

In the standard training path, the student is loaded in 4-bit NF4, prepared for k-bit training, and updated with a persistent LoRA adapter of rank 8 and alpha 16 on q_proj and v_proj. The configured learning rate is 2e-4, per-device batch size is 1, gradient accumulation is 4, sequence length is capped at 512 tokens, and FP16 is enabled. The maximum step count is 15 for fewer than 60 retained examples; otherwise it is based on retained-set size divided by four. The adapter path is reused across runs. The project also has a distinct CS10 branch that bypasses HES and performs full FP16 fine-tuning on hard-coded noise strings; it is an exception and not the standard LoRA path.

4.3 Architecture framework

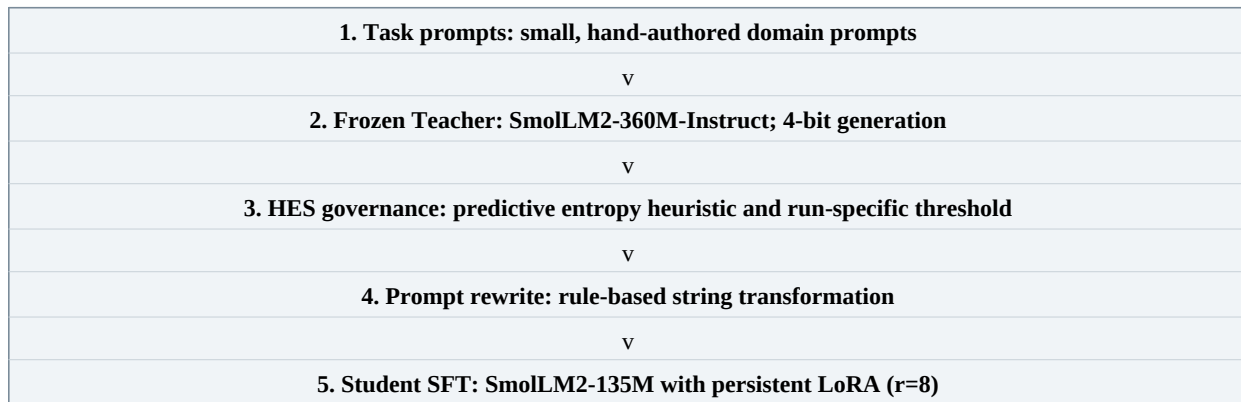


Figure 1. Standard pipeline. In standard runs the student base weights are frozen; CS10 is a separate full-fine-tuning exception.

4.4 HES definition and limitations

At generation step t , let $p_t(v)$ be the softmax probability of vocabulary token v . The code computes token entropy $H_t = -\sum_v p_t(v) \log_2 p_t(v)$. It sorts the T token entropies in ascending order, sets the cutoff index to $\text{floor}(0.8T)$, and sums the values in the slice from that index to the end. This is the implemented HES. It is a predictive-uncertainty heuristic, not a measure of correctness, usefulness, diversity, or retention. Because it sums token entropies, its value also depends on output length. The rule is not uniform across project runs: the standard evaluator uses $\text{HES} \geq 10.0$, CS9 deliberately inverts the threshold at 5.0, CS10 bypasses governance, and standalone main.py uses 15.0.

4.5 Data and evaluation

The case-study entry point uses a few hand-authored prompts in domains including algebra, language, history, creative writing, adversarial logic, coding, and general knowledge. In standard runs, target answers are generated by the teacher at execution time and held in memory. The code contains fixed reference answers for three algebra questions and two empathy questions. The evaluator masks prompt tokens when computing answer loss, weights loss by answer-token count, exponentiates mean loss to obtain perplexity, and writes an `evaluation_results.json` summary when run. That file is absent from the audited project snapshot. The evaluator skips NaN losses but does not log the skipped count or filter positive infinity; future results require per-example records and complete accounting.

A rigorous follow-up should define a versioned stream of disjoint tasks with separate training and held-out examples; score every task before adaptation and after every update; use task-appropriate exact match, accuracy, or blinded rubric scoring, with answer-only cross-entropy as a secondary measure; and preserve sample counts, confidence intervals, failed-example counts, model revisions, seeds, and adapter hashes. Compare (a) a single-model sequential-update baseline, (b) persistent LoRA, and (c) the full dual-SLM pipeline under matched data and update budgets. Include no-update and replay or task-specific-adapter controls as separate arms, and repeat across multiple random seeds. Report new-task learning and prior-task retention separately, including average forgetting relative to each task's best earlier score.

4.6 Reproducibility and ethics

Trainer states and model artifacts are present for Case Studies 1-11 and 13. The master log contains selected examples rather than a complete record of generated outputs, entropy vectors, and run settings. Exact model revisions, full run logs, random seeds, hardware inventory, and historical per-example evaluation records are not consistently preserved. No human participants or personal datasets were used in the described synthetic prompt workflow. Future releases should document model licenses, prompts, safety review for generated training data, software versions, seeds, and evaluation exclusions.

5. Results and Evidence

The code and saved artifacts support several engineering observations. The current implementation contains a separate teacher-generation stage, a governance function, rule-based prompt rewriting, a persistent LoRA student path, and a separate full-fine-tuning exception. Checkpoints and trainer states are present for Case Studies 1-11 and 13. This supports that training activity took place; it does not prove that each case was an independent, controlled experiment or that earlier-task performance was retained.

The CS10 trainer state records a finite training-loss spike: loss begins at approximately 6.61, reaches approximately 77.27 at step 4, and ends at approximately 23.70. Token accuracy reaches zero at the spike and later recovers only partially. This indicates severe training disruption during the full-fine-tuning noise run. It does not establish NaN gradients, infinite weights, or total representational collapse. The case-study document reports extremely large baseline perplexities, but the project snapshot lacks a saved evaluation-results file and a tensor-by-tensor finite-value scan; those claims are unverified and are not treated as findings here.

The evaluator's reference set is limited to three algebra examples and two empathy examples. Although the code can calculate answer-token-weighted loss and perplexity, saved historical records needed to reproduce case-by-case outcomes are absent. This small fixed set can function as an engineering smoke-test, but cannot establish broad retention or statistically reliable mitigation of catastrophic forgetting. Training loss on current synthetic examples measures fit to those examples, not retention on earlier tasks.

Case Study 13 has a saved training artifact, but its reported perplexities and semantic-generalization conclusion cannot be independently confirmed from the current evaluation records. The rewriting stage demonstrates deterministic string transformation in code; it does not demonstrate that the student learned semantic invariance. Stable frozen base weights show preservation of those base parameters in standard LoRA runs, while the persistent adapter remains subject to interference.

6. Expected Results and Outcomes

The observed outcome is an implemented, resource-conscious pipeline with checkpointed training activity and a documented evaluation gap. No validated retention benefit is reported. The expected scientific outcome of the proposed follow-up is a quantitative estimate of whether teacher-student decoupling, HES filtering, and persistent LoRA affect (i) acquisition on each new task, (ii) retention on previous tasks, and (iii) compute and memory use under controlled budgets. The result may support, qualify, or reject the mitigation hypothesis; the design does not presume a favorable outcome.

A useful success criterion should be registered before the follow-up. For example, authors could define an acceptable non-inferiority margin for prior-task performance, report mean and per-task forgetting across seeds, and require that the lower confidence bound meet the registered margin while new-task performance remains comparable. The margin and metric should be selected before observing results.

7. Conclusion and Future Scope

This project implements a dual-SLM pipeline for continual synthetic-data SFT using a frozen SmolLM2 teacher, predictive-entropy heuristic filtering, literal prompt rewrites, and a persistent student LoRA adapter. The frozen teacher provides a fixed parameter reference during a run, and standard LoRA training leaves student base weights unchanged. These engineering properties motivate a hypothesis that the design could reduce some forms of interference, but they do not show that the adapted system avoids forgetting. The adapter itself is continually updated, HES is not a retention objective, and the current five-item reference evaluation is too small and insufficiently archived to establish efficacy.

Future work should implement per-update held-out evaluation, preserve complete run and evaluation ledgers, verify each HES branch, conduct matched comparisons against single-model updates and retention controls, and repeat experiments across seeds. Until those measurements exist, the accurate claim is that this is a functioning prototype and an experimentally testable candidate approach to continual adaptation, not a demonstrated solution to catastrophic forgetting or a refutation of SEAL.

References

- [1] M. McCloskey and N. J. Cohen, “Catastrophic interference in connectionist networks: The sequential learning problem,” *Psychology of Learning and Motivation*, vol. 24, pp. 109-165, 1989. doi: 10.1016/S0079-7421(08)60536-8.
- [2] H. Shi et al., “Continual Learning of Large Language Models: A Comprehensive Survey,” arXiv:2404.16789, 2024. <https://arxiv.org/abs/2404.16789>.
- [3] A. Zweiger, J. Pari, H. Guo, Y. Kim, and P. Agrawal, “Self-Adapting Language Models,” *Advances in Neural Information Processing Systems* 38, 2025. https://papers.neurips.cc/paper_files/paper/2025/file/6b41e04c41726e2a60e456d0a2b961ab-Paper-Conference.pdf.
- [4] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” *International Conference on Learning Representations*, 2022. arXiv:2106.09685. <https://arxiv.org/abs/2106.09685>.
- [5] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” *Advances in Neural Information Processing Systems* 36, 2023. arXiv:2305.14314. <https://arxiv.org/abs/2305.14314>.
- [6] L. Ben Allal et al., “SmolLM2: When Smol Goes Big -- Data-Centric Training of a Small Language Model,” arXiv:2502.02737, 2025. <https://arxiv.org/abs/2502.02737>.
- [7] C. E. Shannon, “A mathematical theory of communication,” *Bell System Technical Journal*, vol. 27, no. 3, pp. 379-423, 1948. doi: 10.1002/j.1538-7305.1948.tb01338.x.